ЛАБОРАТОРНАЯ РАБОТА №5

Цель работы.

Изучение различных фильтров, предназначенных для детектирования границ и улучшения качества изображения

1. Теория матричных фильтров

Матрица свёртки – это матрица коэффициентов, которая «умножается» на значение пикселей изображения для получения требуемого результата.(Рис1.)

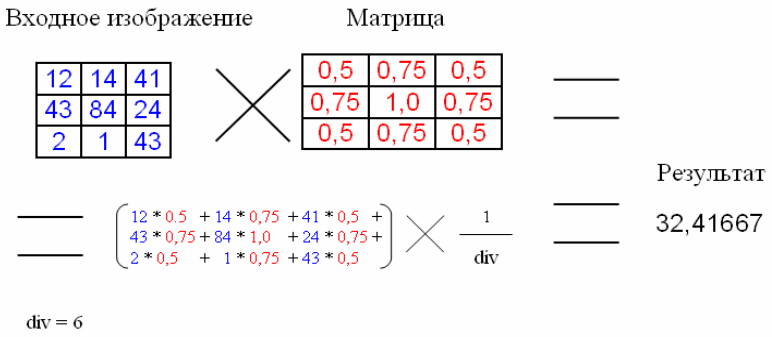


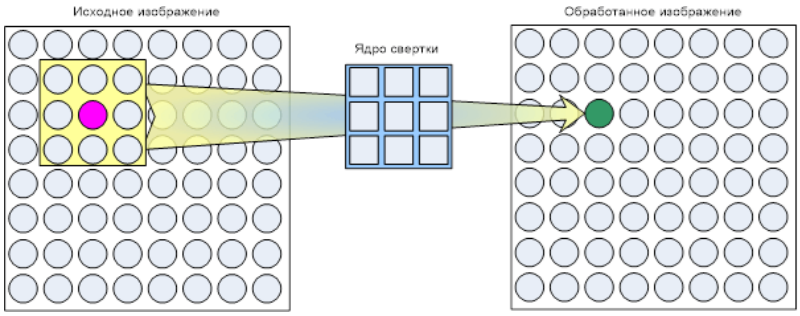
Рисунок1. Пример матрицы свертки.

div – это коэффициент нормирования, для того чтобы средняя интенсивность оставалась не изменой.

1.1. Фильтр размытия и повышения резкости

Наиболее часто используемым фильтром, основанным на матрице свёртки, является фильтр размытия. Обычно матрица заполняется по нормальному (гауссовому закону). Имеет сложность O(hi * wi * n *n), где hi, wi – размеры изображения, n – размер матрицы (ядра фильтра). Квадратное ядро (матрицу) можно заменить двумя одномерными: горизонтальным и вертикальным. Коэффициенты уже являются нормированными, так что div для этой матрицы равен одному.

Эти фильтры реализуются на основе ядра свертки. Элемент изображения получает новое значение на основе группы элементов, примыкающих к данному. Область примыкания есть квадратная матрица, размерность которой совпадает с размером выбранного ядра свертки, и центром в обрабатываемом элементе.



Ядро свертки представляет собой матрицу размером 3x3, 5x5, 7x7 и т.д., на которой определена некоторая функция. Ядро свертки называется окном, а заданная на нем функция – весовой или функцией окна. Каждому элементу окна соответствует число, называемое весовым множителем. Совокупность всех весовых множителей и составляет весовую функцию. Нечетные размеры окна необходимы для однозначного определения центрального элемента.

По сути, ядро свертки является фильтром, который позволяет усилить или ослабить компоненты изображения. Фильтрация осуществляется перемещением окна фильтра по изображению. Весовая функция в процессе перемещения остается неизменной. В каждом положении окна происходит операция свертки. При каждом положении окна весовая функция поэлементно умножается на значение соответствующих пикселей исходного изображения и произведения суммируются. Полученная сумма называется откликом фильтра и присваивается тому пикселю нового изображения, который соответствует положению центра окна.

Результат обработки пикселя записывается в соответствующую ячейку временной матрицы такого же размера, как и исходное изображение. Запись в отдельную временную матрицу необходима для того, чтобы исключить влияние уже обработанных пикселей на еще не обработанные.

При использовании алгоритма увеличения резкости подчеркиваются различия между цветами смежных пикселей и выделяются незаметные детали. В ядре резкости центральный коэффициент больше 1, а окружен он отрицательными числами, сумма которых на единицу меньше центрального коэффициента. Таким образом, увеличивается любой существующий контраст между цветом пикселя и цветами его соседей. При обработке каждого пикселя в изображении используется ядро резкости размерами 3×3:

$$\begin{pmatrix} -1 & -1 & -1 \\ -1 & 9 & -1 \\ -1 & -1 & -1 \end{pmatrix}$$

При использовании алгоритма размытия в изображении перераспределяются цвета и смягчаются резкие границы. Ядро сглаживания состоит из совокупности коэффициентов, каждый из которых меньше 1, а их сумма составляет 1. Это означает, что после фильтрации каждый пиксель поглотит что-то из цветов соседей, но полная яркость изображения останется неизменной. При обработке изображения используется следующее ядро свертки:

$$\frac{1}{9}\begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

1.2 Граничные условия

Стоит упомянуть о граничных условиях (эта проблема актуальна для всех матричных фильтров). У верхнего левого пикселя не существует «соседа» с права от него, следовательно, нам не на что умножать коэффициент матрицы. (Рис. 3)

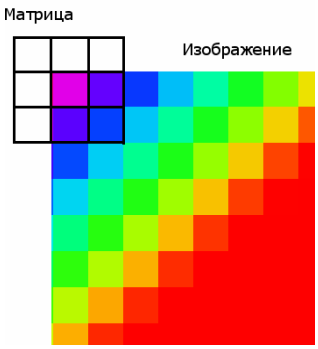


Рисунок 3. Проблема граничных условий.

Для решения этой проблемы требуется создание промежуточного изображения. Идея в том, чтобы создавать временное изображение с размерами (width + 2 • gap / 2, height + 2 • gap / 2, где width и height – ширина и высота фильтруемого изображения, а gap – размерность матрицы свертки). В центр изображения копируется входная картинка, а края заполняются крайними пикселями изображения. Размытие применяется к промежуточному буферу, а потом из него извлекается результат. Рис.4

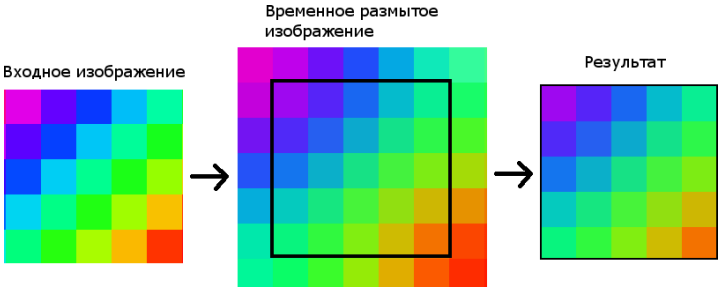


Рисунок 4. Промежуточный буфер.

Данный метод не имеет недостатков в качестве, но необходимо производить лишние вычисления.

1.3 Выделения контуров изображений

- Фильтр Робертса

Пусть область 3x3, показанная на рисунке ниже (см. рис. 2), представляет собой значения яркости в окрестности некоторого элемента изображения.

z_1	z_2	z_3
z_4	z_5	z_6
z_7	z_8	z_9

Рисунок 5. Окрестность 3x3 внутри изображения

Один из простейших способов нахождения первых частных производных в точке image состоит в применении следующего перекрестного градиентного оператора Робертса:

$$G_x = (z_9 - z_5)$$

$$G_y = (z_8 - z_6)$$

Эти производные могут быть реализованы путем обработки всего изображения с помощью оператора, описываемого масками на рисунке 6.

-1	0
0	1

0	-1
1	0

Рисунок 6. Маски оператора Робертса

Реализация масок размерами 2x2 не очень удобна, т.к. у них нет четко выраженного центрального элемента, что существенно отражается на результате выполнения фильтрации. Но этот «минус» порождает очень полезное свойство данного алгоритма – высокую скорость обработки изображения.

- Фильтр Превитта

Оператор Превитта, так же как и оператор Робертса, оперирует с областью изображения 3x3, представленной на рисунке 2, только использование такой маски задается другими выражениями:

$$G_x = (z_7 + z_8 + z_9) - (z_1 + z_2 + z_3)$$

$$G_y = (z_3 + z_6 + z_9) - (z_1 + z_4 + z_7)$$

В этих формулах разность между суммами по верхней и нижней строкам окрестности 3x3 является приближенным значением производной по оси x, а разность между суммами по первому и последнему столбцам этой окрестности – производной по оси y. Для реализации этих формул используется оператор, описываемый масками на рисунке 7, который называется оператором Превитта.

-1	-1	-1
0	0	0
1	1	1

-1	0	1
-1	0	1
-1	0	1

Рисунок 7. Маски оператора Превитта

- Фильтр Собеля

Оператор Собеля тоже использует область изображения 3x3, отображенную на рисунке 2. Он довольно похож на операторПревитта, а видоизменение заключается в использовании весового коэффициента 2 для средних элементов:

$$G_x = (z_7 + 2z_8 + z_9) - (z_1 + 2z_2 + z_3)$$

$$G_y = (z_3 + 2z_6 + z_9) - (z_1 + 2z_4 + z_7)$$

Это увеличенное значение используется для уменьшения эффекта сглаживания за счет придания большего веса средним точкам.

Маски, используемые оператором Собеля, отображены на рисунке 8.

-1	-2	-1
0	0	0
1	2	1

-1	0	1
-2	0	2
-1	0	1

Рисунок 8. Маски оператора Собеля

Рассмотренные выше маски применяются для получения составляющих градиента G_x и G_y. Для вычисления величины градиента эти составляющие необходимо использовать совместно:

$$\nabla f \approx |G_x| + |G_y| \quad \text{ИЛИ} \quad f = \sqrt{G_x^2 + G_y^2}$$

Т.к. значения могут быть отрицательными.

2. Программный комплекс

Приложение разработано на языке C# в среде разработки MicrosoftVisualStudio (версия Community 2019).

Пользовательский интерфейс представлен на рис. 9.

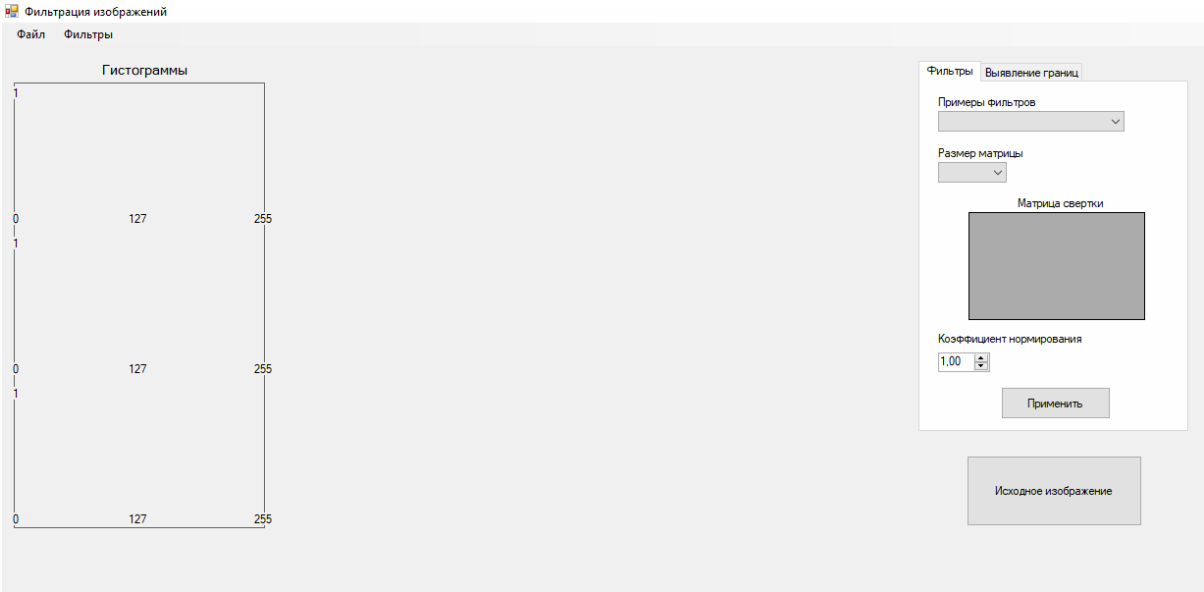


Рисунок 9.

На рисунках ниже представлено главные меню программы. Во вкладке «Файл» находятся кнопки для открытия и сохранения изображения.

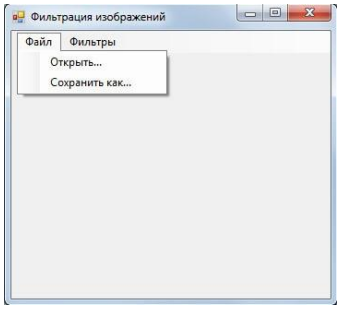


Рисунок 10

В правой части интерфейса представлены инструменты для фильтрации загруженного изображения.

Во вкладке «Фильтры» находятся: Фильтр размытия, повышения резкости. Можно увидеть матрицу свертки, коэффициент нормирования и размер матрицы. Так же если нужно можно подкорректировать эти параметры.Кнопка «применить» запускает процесс фильтрации изображения.

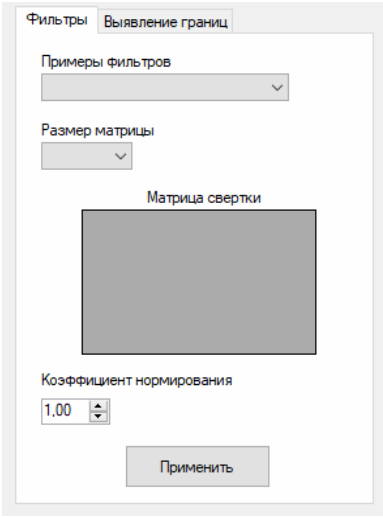


Рисунок 11

Во вкладке «выявления границ» находятся фильтры выявления контуров изображения, описанные выше. Можно увидеть маски, используемые операторами (Gxи Gy), предел и размер ядра. Так же если нужно можно подкорректировать эти параметры.Кнопка «применить» запускает процесс фильтрации изображения.

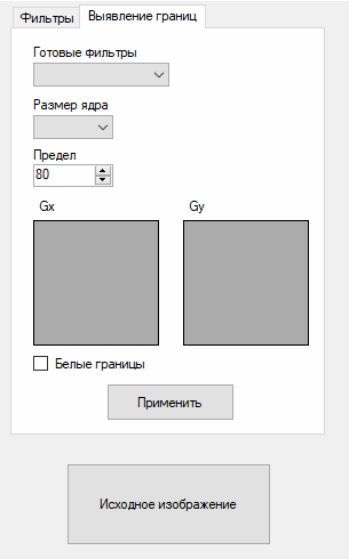


Рисунок 12

Кнопка «Исходное изображение» возвращает картинку к первоначальному виду.

В левой части интерфейса представлены Гистограммы RGBизображения.

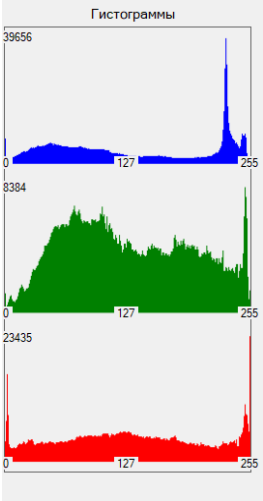


Рисунок 13

3. Пример работы программы

3.1 Фильтр размытия и резкости.

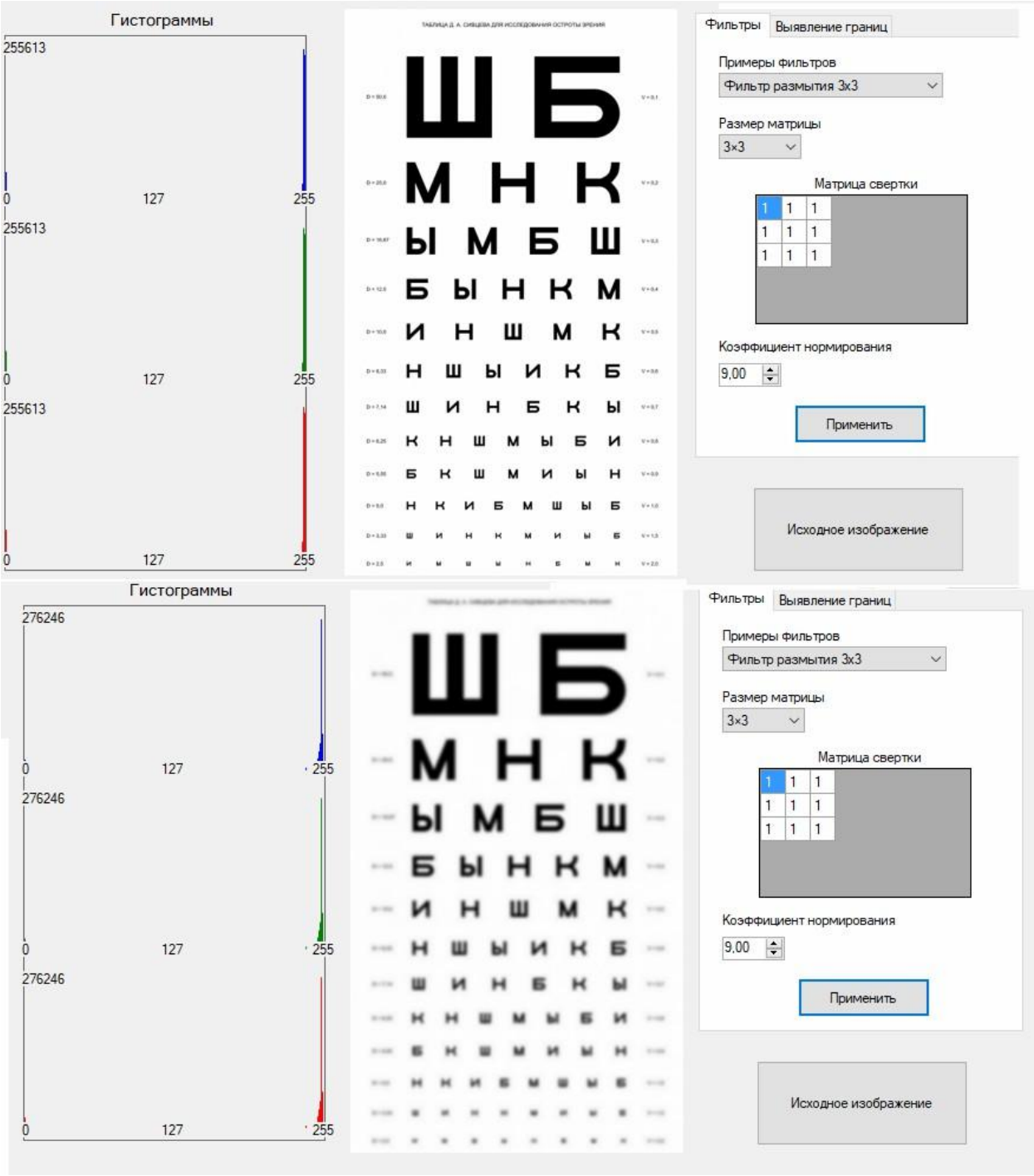


Рисунок 14. Применяем фильтр размытия.

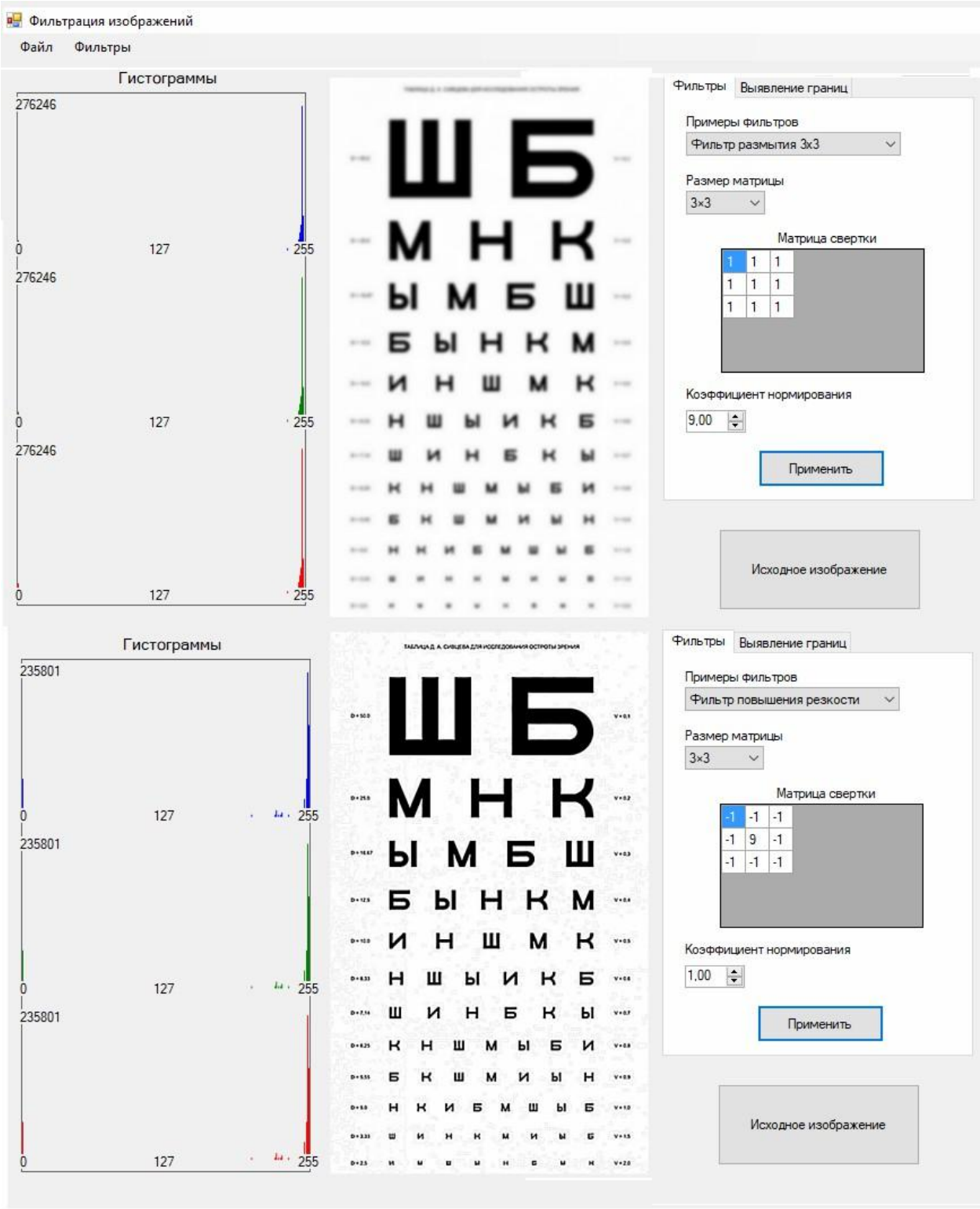


Рисунок 15. Применяем фильтр увеличения резкости.

Рисунок 16. Применяем фильтр улучшения резкости для распознания номера авто.

Благодаря фильтру можно уже различить буквы в номере ТС.

3.2 Фильтры выявления границ

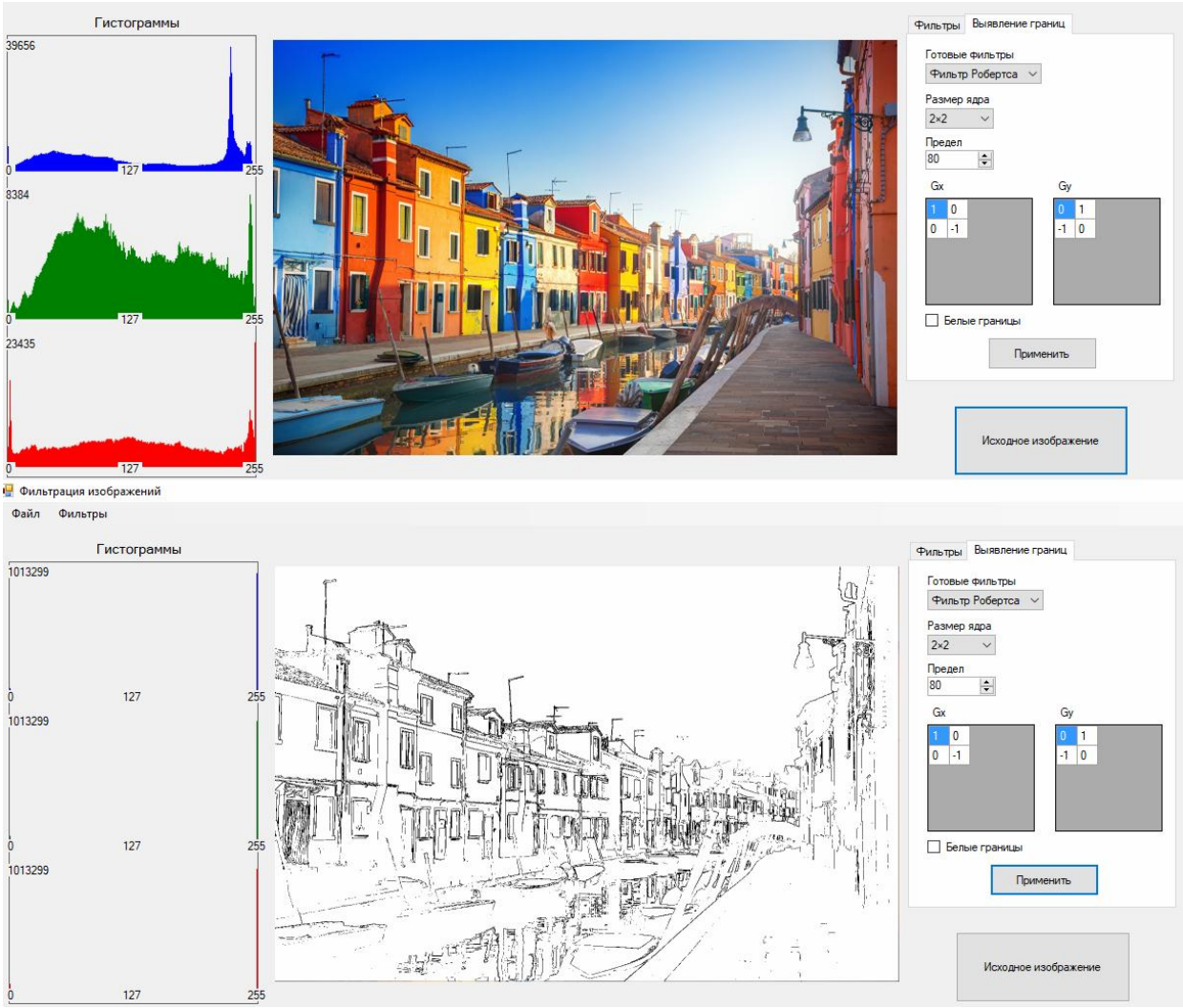


Рисунок 17. Фильтр Робертса.

Рисунок 21. Фильтр Робертся (предел= 30)

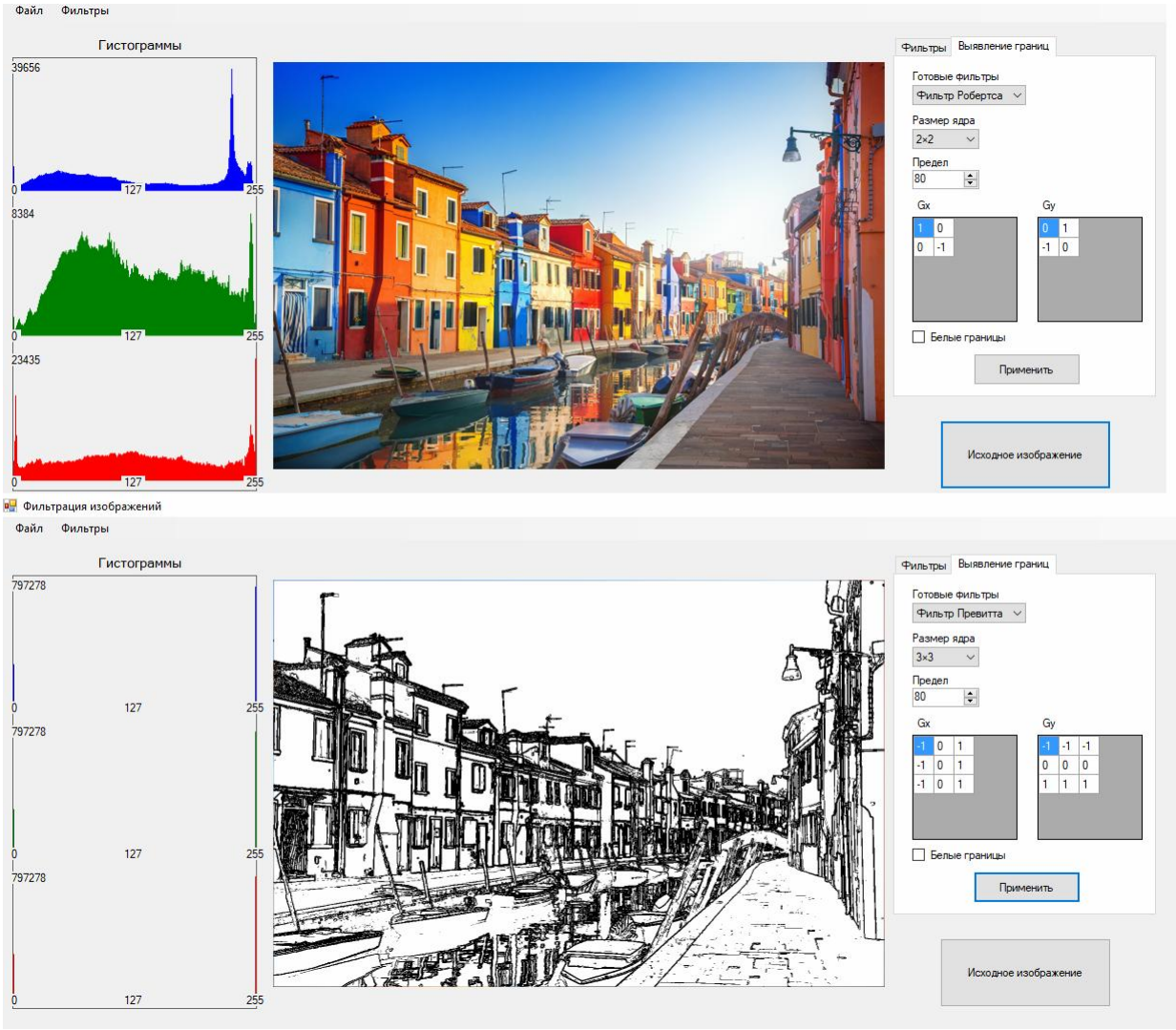


Рисунок 18. Фильтр Превитта.

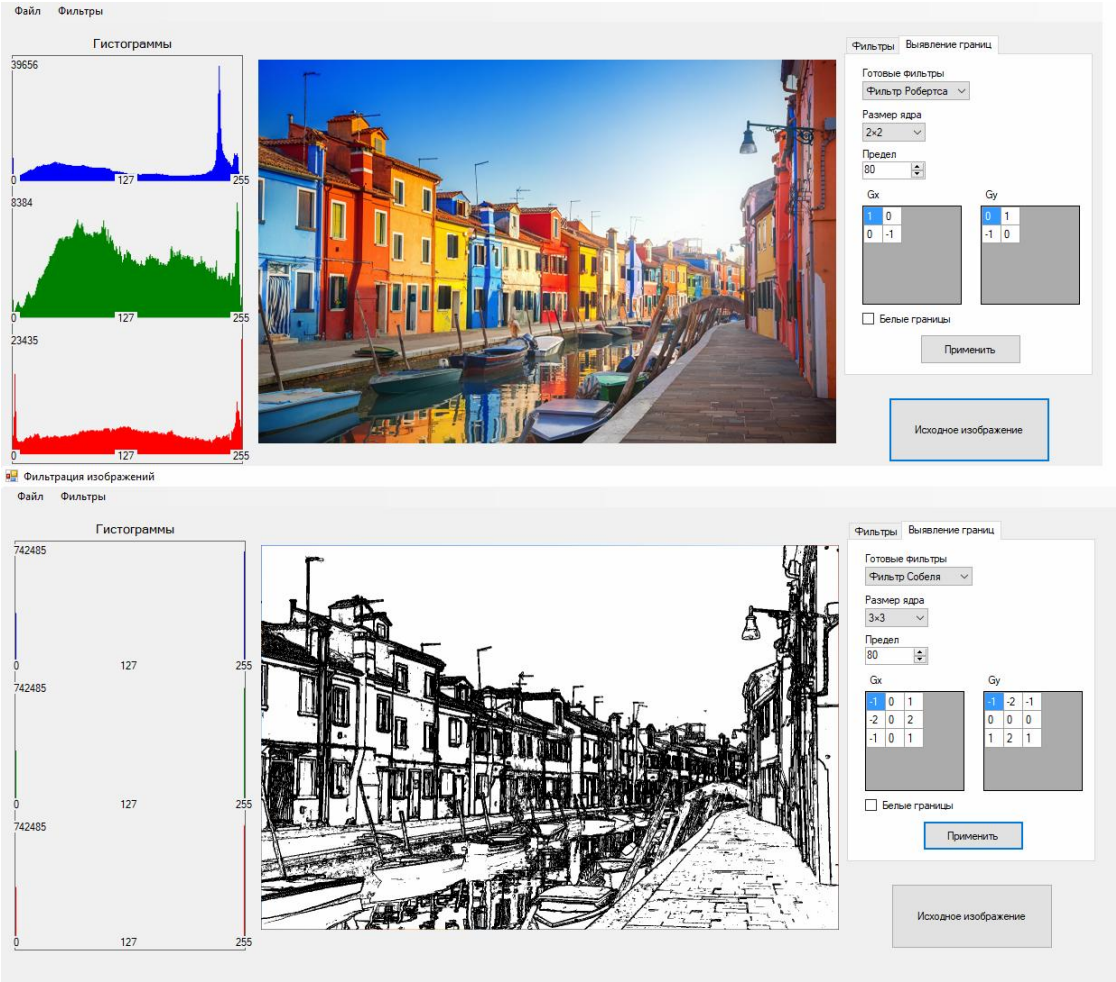


Рисунок 19. Фильтр Собеля.

4. Задание

4.1. Изучить программу и ее воможности.

4.2. Изучить особенности матричных фильтров для выявления границ и размытия/повышения контрастности изображения.

4.3. Функция Canny для выявления границ.

Задание: Довести до рабочего состояния программу, приведенную ниже.

```
import cv2

# Загрузка изображения в оттенках серого
img = cv2.imread('image.jpg',0)

# Применение алгоритма Canny для обнаружения границ
edges = cv2.Canny(img,100,200)

# Вывод изображения с границами
cv2.imshow('Edges',edges)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Параметры функции Canny(image, `threshold1, threshold2, apertureSize`):

1. `image` - входное изображение для обработки. Обычно это изображение в оттенках серого, хотя функция также может работать с цветными изображениями.
2. `threshold1` и `threshold2` - пороговые значения для обнаружения границ. Canny использует два порога (minVal и maxVal), чтобы отфильтровать небольшие границы. Любые границы с интенсивностью пикселей больше maxVal считаются "сильными", а те, которые ниже minVal, отфильтровываются. Границы, которые лежат между этими двумя порогами, считаются "слабыми", и они будут включены в вывод только в том случае, если они связаны с "сильными" границами.
3. `apertureSize` - размер ядра Собеля для вычисления градиента. Этот параметр необязателен, и по умолчанию стоит 3.

4.4 Провести 2-3 эксперимента с разными значениями параметров (порогов) функции Canny().

4.5 Сделать выводы по работе.